

`io.dataflint`

DataFlint

O drop-in replacement do Spark UI que transforma métricas cruas de stages em **alertas de performance nomeados e acionáveis**.

O que vamos cobrir

01

O que é DataFlint

Camada de observabilidade · diagnostica, não corrige · como e onde roda

02

Instalação

Pacote & versão · sessão live · History Server · event logs

03

Alertas & triagem

Os 14 alertas oficiais · disciplina de triagem · roteamento

04

Operação

Versão & Scala · telemetria · segurança · anti-padrões

01

O que é DataFlint

A camada que olha o Spark por você — e te diz, por nome, o que está errado.

Uma camada de observabilidade — não um tuning engine

Um plugin do Spark ([spark.plugins](#)) que adiciona uma aba ao Spark UI existente.

// lê

O **mesmo stream** de eventos e métricas que o Spark UI usa.

// não muda

Não altera a execução. **Zero impacto** no query plan.

// onde

Funciona **live** (driver UI) e **offline** (History Server).

01 · O QUE É DATAFLINT

DIAGNOSTICA

DataFlint encontra

Identifica e nomeia o problema de performance a partir das métricas.

CORRIGE

0 specialist resolve

O fix é aplicado pelo specialist de Spark dono daquele problema.

“DataFlint finds it; the specialists fix it.”

Seis recursos oficiais

01

Status em tempo real

Estado das queries e do cluster, ao vivo

02

Query breakdown

Decomposição da query com heat map de performance

03

Application Run Summary

Resumo executivo de cada execução

04

Alertas & sugestões

Os 14 alertas nomeados — o foco desta revisão

05

Identifica falhas

Erro extraído do stack trace, nó que falhou apontado

06

Spark AI Assistant

Assistente para investigar a execução

Como funciona, por dentro



Fails safe: se o plugin der erro no startup, ele loga um warning e deixa a aplicação seguir — o driver roda em thread separada. **Remoção limpa:** apague as duas configs e o job se comporta de forma idêntica.

Onde roda: dois modos

Real-time UI

aba na driver UI durante o job ativo

Local

Standalone

K8s Operator

EMR

Dataprocc

HDInsight

Databricks

History Server

offline, lê event logs já gravados

Local

Standalone

K8s Operator

EMR

Dataprocc

Não: Databricks · HDInsight · nunca num history server *persistente* (não carrega custom providers).

Dois produtos diferentes

PLUGIN OSS · Apache-2.0

O que usamos aqui

In-process no driver / SHS, aba na UI local

Nada sai do ambiente (com telemetria off)

Ativa com duas configs — package + `spark.plugins`

Seguro como caminho padrão

SAAS · SOC 2 TYPE II

Production-Aware AI Agents

Spark MCP server + 4 agentes:

Copilot · IDE

Cluster Agent

Review Agent

Fleet Observab.

Envia metadados de log para fora → opt-in separado,
decisão CRÍTICA.

02

Instalação

É só configuração. Mas o pacote errado faz o plugin falhar em silêncio.

Escolha o pacote certo

Requer **Spark 3.2+** · Scala 2.12 ou 2.13. Combine o artifact com o seu build.

SPARK	SCALA	MAVEN ARTIFACT
3.0 – 3.5.x	2.12	io.dataflint:spark_2.12
3.0 – 3.5.x	2.13	io.dataflint:spark_2.13
4.x	2.13	io.dataflint:dataflint-spark4_2.13
Databricks 17.3+	2.13	io.dataflint:dataflint-spark4-databricks_2.13

Confirme a Scala do build:
`spark-submit --version`

Última release **0.9.9** (mai/2026). Em produção, **pin exato** — nunca range flutuante.

Capability 1 — sessão live (driver UI)

```
# PySpark — o caminho primário
spark = (SparkSession.builder
  # 1) jar (match Scala + pin)
  .config("spark.jars.packages",
    "io.dataflint:spark_2.12:0.9.9"
  )
  # 2) registra o plugin → ativa
  .config("spark.plugins",
    "io.dataflint.spark.SparkDataflintPlugin"
  )
  # 3) opt-out de telemetria
  .config("spark.dataflint.telemetry.enabled", "false")
  .getOrCreate())
```

→ <http://<driver>:4040>

A aba **DataFlint** aparece na driver UI.

⚠ K8S SEM EGRESS

Maven não resolve em pod sem internet. **Bake o jar na imagem** ou pré-stage no MinIO.

Capability 2 — History Server (offline)

Post-mortem: analisa runs já concluídos a partir dos event logs no MinIO.

- 1 Baixe o jar (Scala correto) da release escolhida.
- 2 Adicione ao classpath: em `$SPARK_HOME/jars/` ou via `SPARK_DAEMON_CLASSPATH`.
- 3 Reinicie o History Server.
- 4 Abra um app concluído → a view **DataFlint** renderiza do replay.

```
# A: jar na pasta jars
cp dataflint-spark_2.12-0.9.9.jar \
  "$SPARK_HOME/jars/"

# B: via classpath env
export SPARK_DAEMON_CLASSPATH=\
  "/opt/dataflint/...jar:$SPARK_DAEMON_CLASSPATH"

# restart
stop-history-server.sh && start-history-server.sh
```

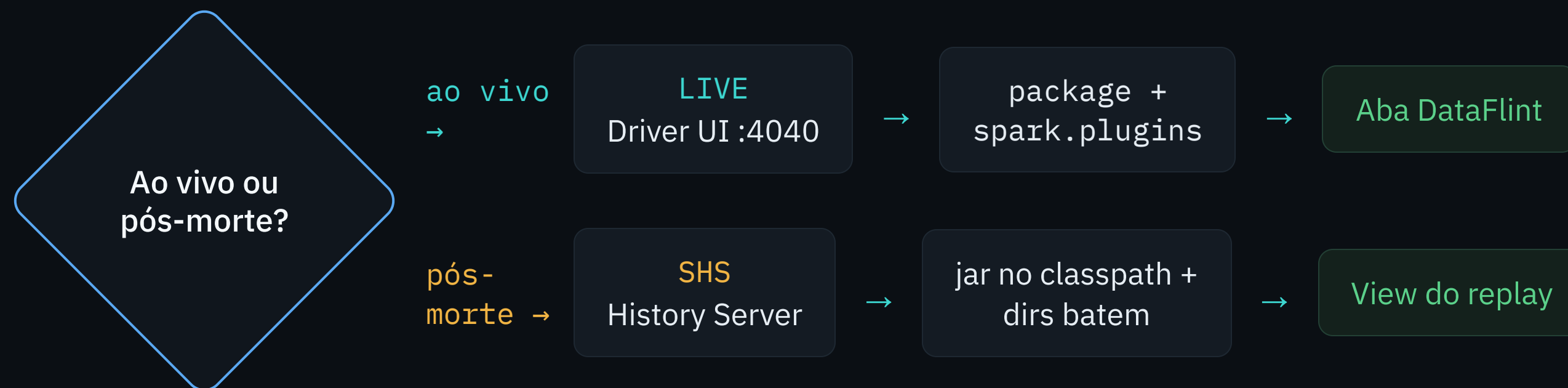

Pré-requisito: os event logs precisam existir

```
spark.eventLog.enabled ..... true
spark.eventLog.dir ..... s3a://<bucket>/spark-events
spark.history.fs.logDirectory .. s3a://<bucket>/spark-events
```



View do DataFlint vazia ≈ SHS vazio. Quase sempre é mismatch entre `eventLog.dir` e `history.fs.logDirectory`, ou credenciais `s3a` do MinIO. **Os dois caminhos têm que bater.**

Fluxo de instalação



⚠ Pod K8s sem egress: jars.packages falha no Maven — **bake o jar na imagem** ou pré-stage no MinIO via spark.jars.

03

Alertas & triagem

DataFlint nomeia o problema. Cabe a você confirmar e rotear.

14 alertas oficiais, em cinco famílias

Nomes verbatim da documentação OSS — cada alerta mapeia para uma causa-raiz provável e um specialist dono do fix.

4

I/O de arquivos

small files, Iceberg,
filtros longos

1

Skew

tarefa muito acima da
mediana

5

Shuffle & joins

tasks, broadcast, cross
join, partições

3

Memória & cores

over/under-provision,
wasted cores

1

Falhas

query failures + stack
trace

Os 14 alertas, pelo specialist que corrige

spark-specialist

Reading Small Files
Writing Small Files
Iceberg — inefficient
replace
Long Filter Conditions
Large Partition Size

spark-skew- specialist

Partition Skew

spark-shuffle- specialist

Large Number Of Small
Tasks
Large Data Broadcast
Broadcast small table in
SMJ
Large Cross Join Scan

spark-manager- specialist

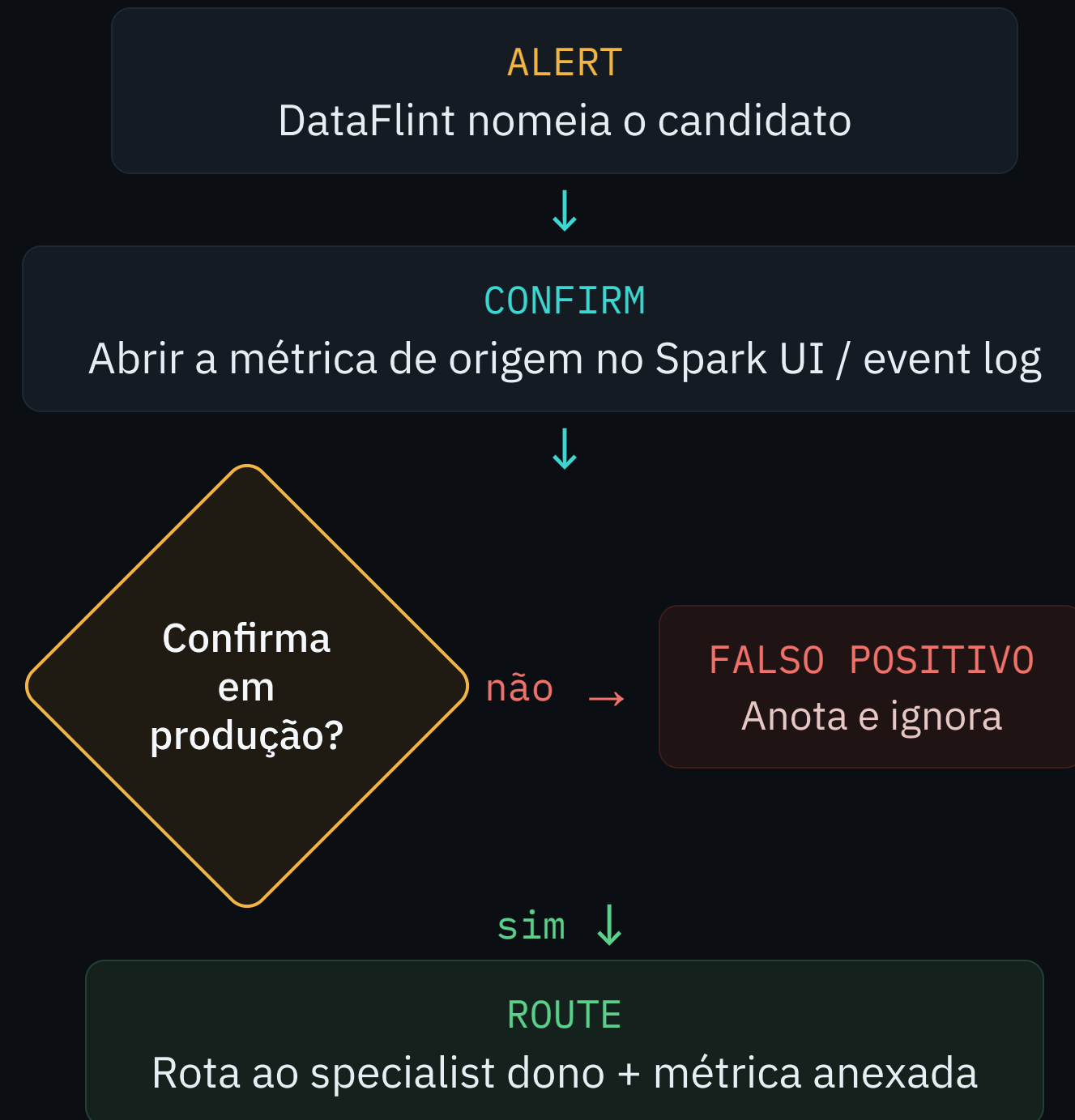
Memory Over-Provisioning
Memory Under-
Provisioning
High wasted cores rate

spark- troubleshooter

Query Failures

Roadmap (ainda não lançados): task/executor error rate · disk spill alto · repartition-before-write de baixa cardinalidade · executor memory overhead baixo.

Disciplina de triagem



Nunca cargo-cult: não mude config só porque o alerta acendeu. Falsos positivos comuns: dataset dev · run frio · coalesce(1) intencional.

Cada eixo tem um dono

`spark-specialist` Arquivos pequenos · Iceberg · partição grande · filtros longos

`spark-skew-specialist` Partition skew → AQE skewJoin ou salting

`spark-shuffle-specialist` Tasks pequenas, broadcast, SMJ, cross join

`spark-manager-specialist` Memória & cores — right-sizing, dynamic allocation

`spark-troubleshooter` Query failures — diagnostica o erro-raiz (OOM, dados, rede)

04

Operação

Versão, telemetria, segurança — e o que nunca fazer.

A falha #1: mismatch Scala / Spark

× ANTI-PADRÃO

`spark_2.13` num build Scala 2.12 → **ClassNotFound** ou o plugin nunca carrega, em silêncio.

✓ FAÇA

Combine o artifact com o banner do `spark-submit --version` e **pin a versão exata**.

Versão flutuante (`io.dataflint:spark_2.12:+`) em produção = builds não reproduzíveis e upgrades-surpresa.

Telemetria & instrumentação

```
# Privacidade – recomendado por padrão aqui
spark.dataflint.telemetry.enabled false

# Métricas de escrita Iceberg (se em uso)
spark.dataflint.iceberg.autoCatalogDiscovery true

# Loga filtros/joins completos (alert Long Filter Conditions)
spark.sql.maxMetadataStringLength 1000

# Instrumentação EXPERIMENTAL de physical plan (default false)
spark.dataflint.instrument.spark.enabled true
```

A instrumentação envolve nós do physical plan (`TimedExec`) para somar métricas de duração que o Spark não reporta — **experimental, ativo com critério**.

Postura de segurança

✓ LOCAL

Roda no driver / history server, no endpoint do Spark UI — **nenhuma porta ou endpoint novo.**

✓ FAILS SAFE

Erro no startup → loga warning e a app segue. Sem compute em executor.

✓ ANÔNIMA

Telemetria coleta só **Spark version + App id** — nunca dados do job. E é desligável.

⚠ SAAS = CRÍTICO

O produto SaaS é uma plataforma separada que envia **metadados de log para fora**. Egress externo = decisão CRÍTICA — nunca sem autorização explícita.

Anti-padrões a evitar

× ANTI-PADRÃO	✓ FAÇA
Jar Scala errado para o build	Match com <code>spark-submit --version</code>
Versão flutuante em produção	Pin de versão exata
<code>jars.packages</code> em pod sem egress	Bake o jar na imagem / stage no MinIO
Aplicar todo alerta como config	Confirmar a métrica, depois rotear
Tratar DataFlint como profiler/fixer	Usar para <i>achar</i> , specialists para <i>corrigir</i>
Telemetria ligada em dado sensível	Setar <code>telemetry.enabled=false</code>
<code>jars.packages</code> no History Server	Sem Ivy no SHS — baixe o jar manual / classpath

Checklist de qualidade

INSTALL

- ☐ Spark ≥ 3.2 confirmado
- ☐ Scala 2.12 vs 2.13 combinada
- ☐ Versão exata pinada
- ☐ Egress K8s tratado (jar baked)
- ☐ SHS: jar + dirs batendo + restart

OPERATE

- ☐ `telemetry.enabled=false` em dado sensível
- ☐ Nenhum egress SaaS sem autorização

TRIAGE

- ☐ Alerta confirmado na métrica real
- ☐ Causa-raiz, não só o sintoma
- ☐ Roteado ao specialist certo
- ☐ Confirmado: execução inalterada

05

DataFlint × Apex

O valor competitivo do Apex — e os gaps que faltam fechar.

Uma comparação assimétrica

DATAFLINT · v0.9.9

Produto entregue

~50 releases · ~466★ · Apache-2.0

Deploys reais — Wix, SimilarWeb, AWS/EMR

Tudo aqui é observado

APEX · v0.1.0 alpha (visado)

Pré-alpha

Primeiros commits · 3 ADRs em revisão

Crew A / DataShip Mission 2026

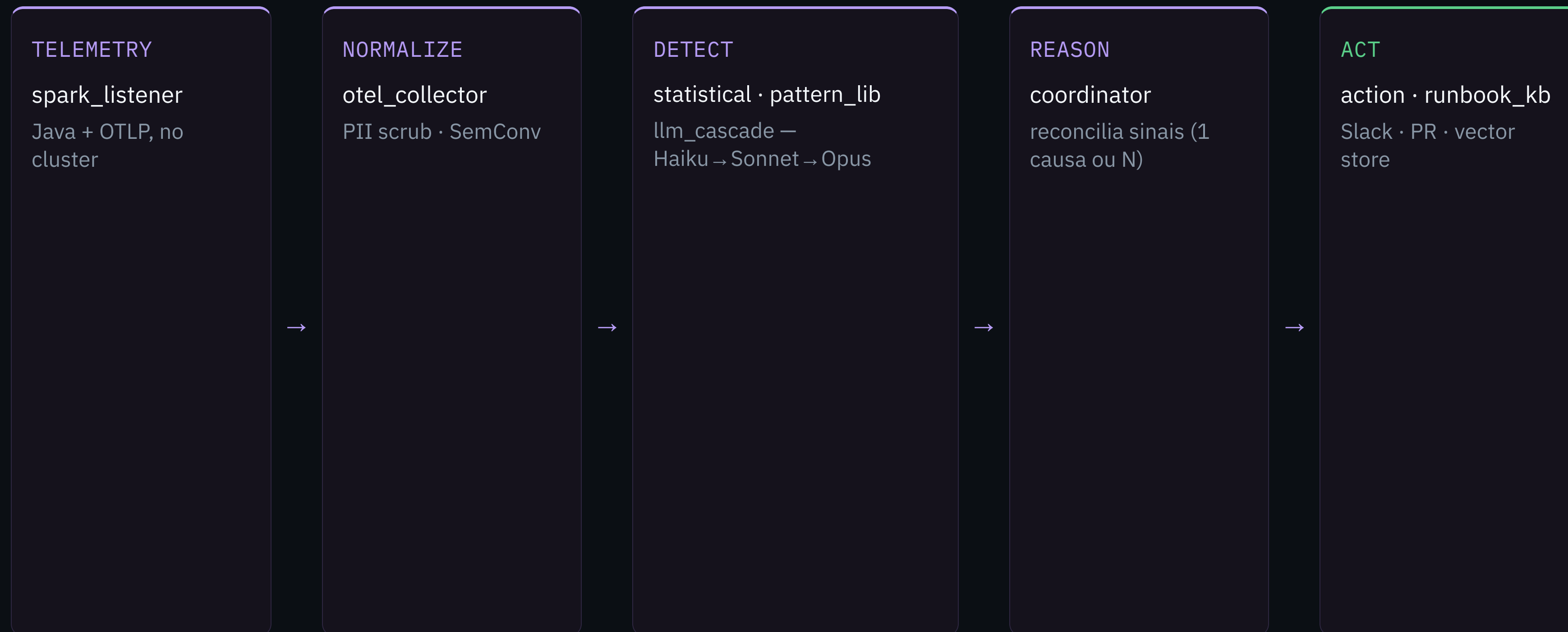
Tudo aqui é planejado / visado

Logo: o valor do Apex é **arquitetural / de visão**; os gaps são de **maturidade / execução**. O DataFlint é, ao mesmo tempo, **concorrente** e **benchmark de referência**.

Matriz de decisão

EIXO	DataFlint OSS	DataFlint SaaS	Apex
Diagnóstico pós-execução	✓ maduro	✓	✓ planejado
Code review no PR (shift-left)	✗	✓ pago	✓ core OSS
Causa-raiz com LLM	✗	✓ pago	✓ cascata
UI visual / heat map	✓ forte	✓	✗ gap
Monitoramento ao vivo	✓	✓	⚠ pós-evento
Catálogo determinístico	✓ 14 provados	✓	⚠ 5 Watchers
Formato Delta / Iceberg	✓	✓	✗ gap
Economia de LLM	n/a	pago	✓ ~95% sem LLM
MCP-native (agent-queryable)	✗	✓	✓ no OSS
Instalação (drop-in)	✓ 2 configs	médio	⚠ pesada
Prova em produção	✓ alta	✓	✗ pré-alpha

Pipeline do Apex — 8 estágios



~95% resolve no T1 determinístico, sem LLM. O Opus (T4) só dispara com confiança <0.6 e severidade ≥ alta — custo é design, não promessa.

Onde o Apex ganha

Shift-left no PR

Revisa o código por AST (≥ 12 anti-patterns) antes de subir. O DataFlint OSS é só reativo.

IA autônoma open-source

Classifier → Coordinator → Judge em Apache-2.0. O DataFlint cobra pela IA (SaaS-only).

Economia de LLM por design

~95% das detecções sem LLM → ~\$200–300/mo vs \$100K+/ano.

Handoff determinístico

Slack · comentário no PR · Runbook KB.
Fecha o ciclo PR → produção.

Cobertura serverless / DLT

OTel + reader out-of-process onde o DataFlint não anexa listener nem roda no SHS.

MCP-native + memória

Consumível por agentes no OSS + Runbook KB (memória institucional).

Gaps que o Apex precisa fechar

P0 · antes do launch

Tira a desvantagem de usabilidade

Read-layer / visual sobre o ClickHouse
(HyperDX / Grafana) — o maior gap

Quickstart 1-container, LLM opcional
(drop-in real)

P1 · constrói o fosso

Onde o Apex realmente vence

Semear a `pattern_lib` com os 14 alertas

Format Watcher (Delta / Iceberg)

Antecipar FR-015 (reader serverless)

OTel streaming p/ visão ao vivo

P2 · maturação

Consolida a proposta

Spark 4.x no listener JVM

MCP server + superfície chat / IDE

Suite de benchmark + pilotos (prova dos 95%)

05 · DATAFLINT × APEX

P0

Visual + install — sair da desvantagem de usabilidade

P1

Shift-left + serverless + formato — construir o fosso

P2

Spark 4.x + chat/IDE + prova — maturar

“O DataFlint te mostra o problema e te faz pagar pela IA que resolve. O Apex é open-source, raciocina a causa-raiz, e pega o problema no PR — **antes de virar incidente.**”

● `io.dataflint`

**“DataFlint finds it;
the specialists fix it.”**

Um read rápido e honesto da saúde do Spark — instalado certo para a realidade Spark / Scala / K8s da plataforma, lido sem cargo-cult, roteado para quem corrige.